



Strojové učení a rozpoznávání

Dokumentace

1 Image classifiers

This section describes the proposed approaches for detecting the target person based on facial images. Due to the limited size of the dataset and the requirement to generalize on unseen data, we implemented a convolutional neural network (CNN) and SVM classifier. In the following subsection, we first describe the CNN-based approach.

1.1 Image classifier based on the CNN

Convolutional neural networks are a standard approach for image classification tasks. However, their effectiveness depends on the availability of a large amount of training data. In this project, the dataset is small and imbalanced, making it likely that deep neural architectures will overfit.

For this reason instead of using some standard deep networks as VGG or ResNet we introduced custom CNN. Firstly, we wanted to use custom CNN inspired by ResNet architecture, but it was achieving poor results. The final custom CNN architecture was inspired by last years network for a similar task, which was simplified and adapted to the binary classification task. The architecture consists of three convolutional blocks, each composed of convolutional layer, batch normalization, ReLU activation, and pooling layer.

To reduce overfitting, dropout regularization is applied in both feature extraction and classification parts of the network. The final classifier consists of two fully connected layers. For classification, two output softmax is used, where each neuron represents one class. As loss function, we used cross entropy.

Dataset

For data splitting, we used a 3-fold cross-validation strategy to increase the number of training data. Specifically:

- Target samples were split based on recording sessions, ensuring that samples from the same session did not appear in both the training and the validation sets.
- Non-target samples were split based on speaker identity, ensuring that identities in the validation set are not present in training data.

To further increase the number of data and improve robustness, we used data augmentation. The transformations used were as color jitter, gaussian blur, random cropping, and various rotations. The augmentations were kept moderate, as aggressive transformations lead to worse performance.

Training and Evaluation

The model was trained using stochastic gradient descent. A OneCycle learning rate scheduler was used to improve convergence. To address class imbalance, a weighted sampling strategy was used during training, ensuring balanced representation of target and non-target samples.

The model was trained for 40 epochs. Three models were obtained from the cross-validation process, and the final prediction is computed as the average output of all three models. We do not have the achieved accuracy on the test data because we decided to use all data in training. The model design and hyperparameter tuning were guided by cross-validation performance, confusion matrices, and monitoring of training and validation loss to avoid overfitting.

Analysis of model outputs showed that non-target samples typically received low confidence scores, while target samples achieved scores close to 0.5. Based on this observation, a decision threshold of 0.4 was selected for the evaluation data to improve target detection. All experiments were conducted using Google Colab, and the implementation of this approach is provided in the form of a Jupyter notebook. In the next subsection, we will describe the second approach for image classification based on SVM.

1.2 Image classifier based on the SVM

For the second approach, we used a Support Vector Machines (SVM) classifier. This approach was selected because it is less prone to overfitting on small datasets compared to CNNs.

There are two possible approaches for training the SVM classifier. The first approach is to use raw pixel values directly, reduce the dimensionality using PCA, and then train the classifier. The second approach is to extract feature representations from images and train the SVM on these features. We decided to use feature extraction, where the raw images were converted into feature vectors using Histogram of Oriented Gradients (HOG). The HOG descriptors capture local edge and shape information, which is suitable for face classification tasks.

First, the features are normalized using StandardScaler. Then, Principal Component Analysis (PCA) is applied to reduce dimensionality while preserving 90 % of the variance. Finally, an SVM with an RBF kernel is used as the classifier. We also experimented with alternative configurations, including removing PCA and replacing it with Linear Discriminant Analysis. However, the best performance was achieved with the PCA.

Dataset

In this approach, we decided to use a more standard split of the data. We divided the data into a train and a test set. As described in the task, the sets must be split more logically, because when it is done randomly the SVM will overfit on the test data. To tackle this problem, we tested only on the third set for target data, and non-target data were randomly split. To further address class imbalance, class weights were applied during training, giving higher importance to the target class.

We used data augmentation, too, but they needed to be less aggressive even compared to CNN, because the results were significantly degrading. We used augmentations only on training set, including geometric transformations such as small rotations, translations, and brightness adjustments.

Evaluation

Instead of using direct class predictions, the SVM decision function was used to obtain continuous confidence score, which we later use in threshold based decision making, but the threshold was kept on 0, where everything positive is considered as target and everything negative is considered as non-target.

The results of the test data achieved approximately 90% accuracy, with slight variations between runs, around 2.5 %. The accuracy value represents the number of correct classifications divided by the total number of samples. In the following section, we describe the audio classification systems.

2 Audio classifiers

This section describes the solution for detecting the target person based on the voice recordings. For this part, we followed the approach introduced in lectures and implemented a Gaussian Mixture Model (GMM) using Mel-Frequency Cepstral Coefficients (MFCC).

In our implementation, we trained two separate GMMs, one for the target class and the other for the non-target class. Each model consists of 16 Gaussian components. For feature extraction, we used 13 MFCC coefficients with an FFT size of 400 and a hop length of 160. In addition to MFCCs, we also included the first- and second-order derivatives, delta and delta-delta features.

Dataset

We implemented two variants of this system that differ in the way the data are split. The first variant uses a train-test split similar to the SVM approach, while the second uses all available data for training. In the

train-test setup, the target samples were split based on recording sessions, where one session was used for testing. Non-target samples were split randomly.

For training, we used data augmentation only for the system with the train-test data split. The augmentations were degrading the performance of the classifier, so it was necessary to find balanced use of the augmentations and not to do it very aggressively. So for the final evaluation, we decided to use augmentations only in train-test model, because here we could see the performance clearly on the test data, where with the other model it was not so easy to see if it is not too much, when trained on all data. The applied augmentations included adding white noise, random gain, pitch shifting, and time stretching. Augmentation was applied more frequently to target samples to partially compensate for the class imbalance.

Evaluation

During experiments, we tried to change the number of components of each GMM, but the best results were balanced with 16 components. We even tried to double the number of components for non-target GMM when they contain more data, but the results were worse.

Another important thing to do was to remove the silence from the recordings, because the GMM would learn the silence to distinguish between the target and non-target. For this, we used the trim function from the librosa library. It helped the performance a bit. Significant improvement was made when we hard trimmed the first 2 seconds of each recording, as recommended in the lecture. We were even experimenting with the type of covariance matrix. We tried full, tied, and diagonal, where the diagonal achieving the best results.

Overall, this model achieved approximately 92 % accuracy on the test set. For the second variant, no separate test accuracy is reported, as the model was trained using all available data. In the following section we are going to describe the implementation of the combination of the models.

3 Multimodal classifier

For the combination of models, we decided to use the SVM image classifier and the GMM audio classifier. For fusion, we applied a logistic regression model as a score-level fusion method.

We first generated confidence scores from SVM decision scores and log-likelihood ratios from the GMM for audio. These scores were computed on the same dataset used as the evaluation set for both GMM and SVM. Each sample is represented as a pair of values: (image score, audio score) with the corresponding label.

These score pairs were then used to train a logistic regression model, which learns how to combine both modalities into a single decision function. The logistic regression model learns the optimal weighting between the image and audio scores, as well as the bias. During inference, the trained logistic regression model takes the image and audio scores as input and produces the final classification decision.

4 Used technology and reproducibility

The whole project was written in Python 3.11. The required libraries are listed in *requirements.txt* and can be installed by *pip install -r requirements.txt*. We used the following libraries:

- **Numpy** for common mathematics operations
- **Pandas** for data handling
- **PIL** for image processing
- **Os and Pathlib** for dataset handling

- **Pytorch** for training of neural network for image classification
- **Albumentations** for online augmentations during training
- **Scikit-learn** for SVM, logistic regression, and evaluation metrics
- **Scikit-image** for feature extraction (HOG)
- **Librosa** for audio processing and MFCC feature extraction
- **Matplotlib** for visualization and plotting

To run the python scripts, the *target_** and *non_target_** folders must be placed inside the *SRC* directory. For evaluation of the eval data, the folder *./Eval_data/eval* must also be located inside the *SRC* directory. The project consists of several Python scripts that implement different parts of the pipeline:

- **audio_gmm.py** implements the training and evaluation of the GMM audio classifier using a train-test split.
- **audio_gmm_all.py** implements the audio GMM classifier trained on the full dataset without a separate test split.
- **svm.py** implements the SVM image classifier.
- **fuser.py** implements the multimodal fusion model. It requires pretrained SVM and GMM models, as well as the *scores_image.txt* and *scores_audio.txt* files generated by the respective scripts. These score files are used to train the logistic regression fusion model.
- **helper_functions.py** contains shared utility functions used across multiple scripts to avoid code duplication. The file is not directly executable and only provides reusable function definitions.
- **image_cnn.ipynb** contains the CNN image classifier. This implementation was developed and executed in Google Colab using Google Drive for data access and is not guaranteed to run in a local environment without additional configuration.

To execute any script, run:

```
python script_name.py
```